

Technical R&D Audit Report: SynapseFlow API Gateway

Candidate: Armir Pepkola

Role Context: Technical Consulting / R&D Tax Audit

1. Project Overview & Architectural Challenge

The SynapseFlow project was initiated to solve a specific, high-throughput data ingestion bottleneck: standard web-native API gateways (Kong, Express, Fastify) introduce unacceptable memory fragmentation when parsing custom, high-frequency binary telemetry streams (e.g., IoT sensor networks or financial market data).

The R&D objective was to engineer a custom Node.js middleware service capable of ingesting raw binary TCP streams, translating them in real-time, and exposing the data via GraphQL, without suffering from V8 Garbage Collection (GC) pauses under heavy concurrent load.

2. Technological Uncertainties Investigated

The core technological uncertainty involved memory allocation within a single-threaded runtime environment. Specifically: *Could a dynamically typed runtime achieve C++-like binary parsing speeds without event-loop blocking or Out-Of-Memory (OOM) crashes when subjected to 50,000+ concurrent streaming packets?*

3. Iterative Experimentation & Solution

- **Iteration 1 (Naive Buffer Parsing):** Standard per-packet dynamic memory allocation (`Buffer.from()`) resulted in severe V8 heap fragmentation. Consequent "stop-the-world" GC cycles led to extreme latency spikes and eventual container crashes.
- **Iteration 2 (Custom Zero-Copy Slab Allocation):** I engineered a custom memory pooling mechanism. The gateway now pre-allocates a continuous memory slab at initialization (`Buffer.allocUnsafe`). Incoming binary TCP chunks are written directly to memory pointers within this circular buffer, entirely bypassing Node.js object instantiation. Binary-to-JSON payload translation occurs lazily (on-demand) only when requested by the GraphQL layer.

4. Measurable R&D Outcomes & Containerized Benchmarking

To validate the architecture, the solution was packaged into a lightweight Docker container (Alpine Node.js) and subjected to localized stress testing using distributed load generation (autocannon).

The custom zero-copy buffer successfully eliminated GC-induced latency spikes, yielding the following deterministic metrics while concurrently ingesting background TCP streams:

- **Sustained Throughput:** 14,747 avg requests/sec

- **Peak Burst Capacity:** 17,151 requests/sec
- **Median Latency (p_{50}):** 66 ms (Accounted for Docker WSL2 NAT virtual network bridge overhead)
- **Error Rate:** 0.00% (148,000 requests processed successfully)

Commercial Viability: By mitigating memory bloat and stabilizing throughput, this architecture allows organizations to run translation gateways on smaller, memory-optimized cloud instances (e.g., AWS EC2 Graviton2), significantly reducing projected infrastructure scaling costs while maintaining strict SLA compliance.